

RAPPORT SYSTEM EXP

```
pid_t pid;

printf("Père %d: Processus fils non encore créé\n", getpid());

pid = fork();

if (pid != -1) {
    if (pid != 0) {
        printf("Père %d: Création du fils %d\n", getpid(), pid);
    }
    else
        printf("Fils %d: je suis créé par %d\n", getpid(), getppid());
}
else
    printf("Père %d: Création impossible du processus fils\n");

return 0;
}
```

1. Déroulez et expliquez que fait ce programme.
2. Transformez ce programme de sorte que le fils renvoie un résultat à son père et qui sera affiché

kirouane nermine isil B groupe 1

30/04/2024
SYSTEM EXP

Exercice 3 (Création de processus avec fork)

1. Déroulez et expliquez que fait ce programme.

Soit le programme suivant :

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
```

1/3

```
    pid_t pid;

    printf("Père %d: Processus fils non encore créé\n", getpid());

    pid = fork();

    if (pid != -1) {
        if (pid != 0) {
            printf("Père %d: Création du fils %d\n", getpid(), pid);
        }
        else
            printf("Fils %d: je suis créé par %d\n", getpid(), getppid());
    }
    else
        printf("Père %d: Création impossible du processus fils\n");

    return 0;
}
```

```
/home/nermine/.hushlogin file.
nermine@DESKTOP-0112FBV:~$ touch exo3tp3.c
nermine@DESKTOP-0112FBV:~$ nano exo3tp3.c
nermine@DESKTOP-0112FBV:~$ |
```

i created the file.c that will have the program

```
GNU nano 7.2          exo3tp3.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main() {
    pid_t pid;
    printf("Père %d: Processus fils non encore créé\n", getpid());
    pid = fork();
    if (pid != -1) {
        if (pid != 0) {
            printf("Père %d: Création du fils %d\n", getpid(), pid);
        }
    }
}
```

explaining how this program work:

```
nermine@DESKTOP-0112FBV:~$ ./exo3tp3
Père 542: Processus fils non encore créé
Père 542: Création du fils 543
Fils 543: je suis créé par 542
nermine@DESKTOP-0112FBV:~$
```

the program declared a variable type `pid_t`

`printf = Père 542: Processus fils non encore créé`

mean the processus père have the `pid = 542`

`//printf("Père %d: Création impossible du processus fils\n");`

this message never got printed because no error occurred

`pid = fork();`

create from processus parent a processus fils

processus father gonna return pid of the fils

processus fils return 0

that why père second pid printf have the same as the first one because it return vale > 0

and processus fils return 0 so it printf the pid of the new pid of the processus fils

2. Transformez ce programme de sorte que le fils renvoie un résultat à son père et qui sera affiché par ce dernier.

it

```
nermine@DESKTOP-0112FBV: ~/Desktop$ ./ex05.c
Père 630: Processus fils non encore créé
Père 630: Création du fils 631
Fils 631: je suis créé par 630
les résultat de fils est 5nermine@DESKTOP-0112FBV:~$ |
```

after modifying it we can see the status that the fils did exit with

```
pid = fork();
int status;
if (pid != -1) {
if (pid != 0) {
printf("Père %d: Création du fils %d\n", getpid(), pid);
wait(&status);
printf("les résultat de fils est %d" , WEXITSTATUS(status));
}
else
printf("Fils %d: je suis créé par %d\n", getpid(), getppid());
}
```

i declared int status to store the exit value of the program fil

and using WEXITSTATUS = waitexitstatus

we get the status that the program did exit with

using wait to wait for the processus fil (zombie processus) to send the exit number

```
else
printf("Fils %d: je su
exit(5);
}
```

3. Modifiez ce programme pour que le processus fils créé lance la commande ls -l sur le répertoire

courant. (Utilisez une des fonctions de la famille exec)

```

nermine@DESKTOP-0112FBV:~$ gcc exo3tp3.c
nermine@DESKTOP-0112FBV:~$ gcc exo3tp3.c -o exo3tp3
nermine@DESKTOP-0112FBV:~$ ./exo3tp3
Père 682: Processus fils non encore créé
Père 682: Création du fils 683
Fils 683: je suis créé par 682
total 160
-rw-r--r-- 1 nermine nermine  0 Apr 10 08:43 ,2,4,45,410,510,_lunaire
drwxr-xr-x 2 nermine nermine 4096 Apr 10 08:49 Ans
-rw-r--r-- 1 nermine nermine 10240 Apr 21 20:50 Ans.tar
drwxr-xr-x 2 nermine nermine 4096 Apr 22 16:57 Archive
drwxr-xr-x 2 nermine nermine 4096 Apr 22 22:29 NouvelleAns

```

i used the easiest one (execvp)

this take directly the command we wanna execute and adding null to let the program know the end of the program system

```

}
else
printf("Fils %d: je suis créé par %d\n", getpid(), getppid());
execvp("ls" , "ls" , "-l" , NULL);
exit(5);

```

Exercice 4 (Création de processus avec system)

```

int main() {
    int return_value;
    return_value = system("ls -l /");
    return return_value;
}

```

1. Que fait la primitive system ? Vérifiez en remplaçant la commande ls par le programme de l'exercice 1.

the program system fonctionne comme les function (exec famille)

donc le program exécute la command ls -l

it attend la fin de la command et return le code return (exit code)

`system()` = exécuter une commande Linux depuis un programme c

```
nermine@DESKTOP-0112FBV:~$ ./exo3tp3
Père 738: Processus fils non encore créé
Père 738: Création du fils 739
Fils 739: je suis créé par 738
total 160
-rw-r--r-- 1 nermine nermine  0 Apr 10 08:43 ,2,4,45,410,510,-
drwxr-xr-x 2 nermine nermine 4096 Apr 10 08:49 Ans
-rw-r--r-- 1 nermine nermine 10240 Apr 21 20:50 Ans.tar
drwxr-xr-x 2 nermine nermine 4096 Apr 22 16:57 Archive
drwxr-xr-x 2 nermine nermine 4096 Apr 12 22:29 NouveAns
```

it did got execute the same way instead of using the command `exec` (famile functions) i used the command `system` directly

```
printf("les résultat de fils est %d" , WEXITSTATUS(
}
else
printf("Fils %d: je suis créé par %d\n", getpid(), getppid(
system("ls -l");
exit(5);
}
else
```

^G Help	^O Write Out	^W Where Is	^K Cut
^X Exit	^R Read File	^N Replace	^U Paste